



Darstellungsschicht: Probleme der Datenübertragung

Warum muss eine Darstellungsschicht implementiert werden?

- Heterogenität im verteilten System:
 - Heterogenität der Plattform (Betriebssystem, Hardware)
 - Heterogenität der Programmiersprache
- Heterogene Hardware-Architekturen
 - Unterschiedliche Reihenfolge der Speicherung von Bytes: Little Endian / Big Endian
 - Unterschiedliche Zeichencodierung:
 - ASCII auf PCs / EBCDIC auf IBM Mainframes





Problem: Uneinheitliche interne Darstellung

Unterschiedliche Darstellungen des Wortes „JILL“ und der Zahl 5 am Beispiel Little-Endian (Intel) / Big-Endian (SPARC)

Integer: 5
Zeichenkette: JILL

3	2	1	0
0	0	0	5
L	L	I	J
7	6	5	4

Intel Pentium
little endian

Integer: 83886080
Zeichenkette: JILL

0	1	2	3
5	0	0	0
J	I	L	L
4	5	6	7

Empfang auf Sun SPARC
big endian

Integer: 5
Zeichenkette: LLIJ

0	1	2	3
0	0	0	5
L	L	I	J
4	5	6	7

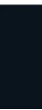
Nach Invertierung
der Bytes

Integer werden durch die unterschiedliche Byteordnung gedreht,
aber Zeichenketten nicht.



Datendarstellung sollte explizit gelöst werden

- Idee:
 - Definiere eine **Menge abstrakter Datentypen** und eine **Kodierung** (ein **genaues Bit-Format**) für jeden dieser Typen
 - Stelle **Werkzeuge** zur Verfügung, die die abstrakten **Datentypen in** Datentypen der verwendeten **Programmiersprache** übersetzen
 - Stelle **Werkzeuge** zur Verfügung, die die Datentypen der verwendeten **Programmiersprache in** die abstrakten **Datentypen** und damit in das kodierte Format übersetzen
 - *Senden (**Marshalling**)*: wenn ein bestimmter Datentyp übertragen werden soll, rufe die Kodierungsfunktion auf und übertrage das Ergebnis
 - *Empfangen (**Un-Marshalling**)*: dekodiere den Bit-String und erzeuge eine neue lokale Repräsentation des empfangenen Typs



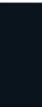


Hinweis

Es gibt eine pragmatisch einfache Lösung zur Implementierung einer Darstellungsschicht:

- Verwende ein (einfaches) textbasiertes Protokoll mit eindeutiger Zeichenkodierung
- Hierbei muss aber sichergestellt werden, dass alle Inhalte auch als Text darstellbar sind

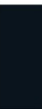
Bekannte Ansätze: HTTP und SMTP (Mail)





Beispiel existierender Formate für eine eigenständige Datendarstellung

- ISO: **ASN.1** (Abstract Syntax Notation)
- Sun **ONC** (Open Network Computing)-RPC: **XDR** (eXternal Data Representation)
- OSF (Open System Foundation)-RPC: **IDL** (Interface Definition Language)
- *Corba*: **IDL** und **CDR** (Common Data Representation): CDR bildet IDL-Datentypen in Bytefolgen ab.
- **XML/SOAP**
- **Java: Objektserialisierung**, d.h. Abflachung eines (oder mehrerer) Objektes zu einem seriellen Format inkl. Informationen über die Klassen.
Deserialisierung ist die Wiederherstellung eines Objektes ohne Vorwissen über die Typen der Objekte
- **JSON** (JavaScript Object Notation)
- ...





ASN.1: Definition

ASN.1 (abstrakte Syntax-Notation eins) ist eine von der ISO genormte Beschreibungssprache zur darstellungsunabhängigen Spezifikation von Datentypen und Werten

- findet z.B. zur Definition von Managementobjekten bei SNMP-Verwendung

- **Elementare Datentypen:**

- Boolean, Integer, Bitstring, Octetstring, IA5String, ...

- **Strukturierte Datentypen:**

- **Sequence:** Geordnete Liste von Datentypen (struct in C)
- **Set:** Ungeordnete Menge von Datentypen
- **Sequence OF:** Geordnete Liste von Elementen des gleichen Datentyps (Array in C)
- **Set OF:** Ungeordnete Menge des gleichen Datentyps
- **Choice:** Ungeordnete Menge von Datentypen, aus der einige Datentypen ausgewählt werden können (Union in C)

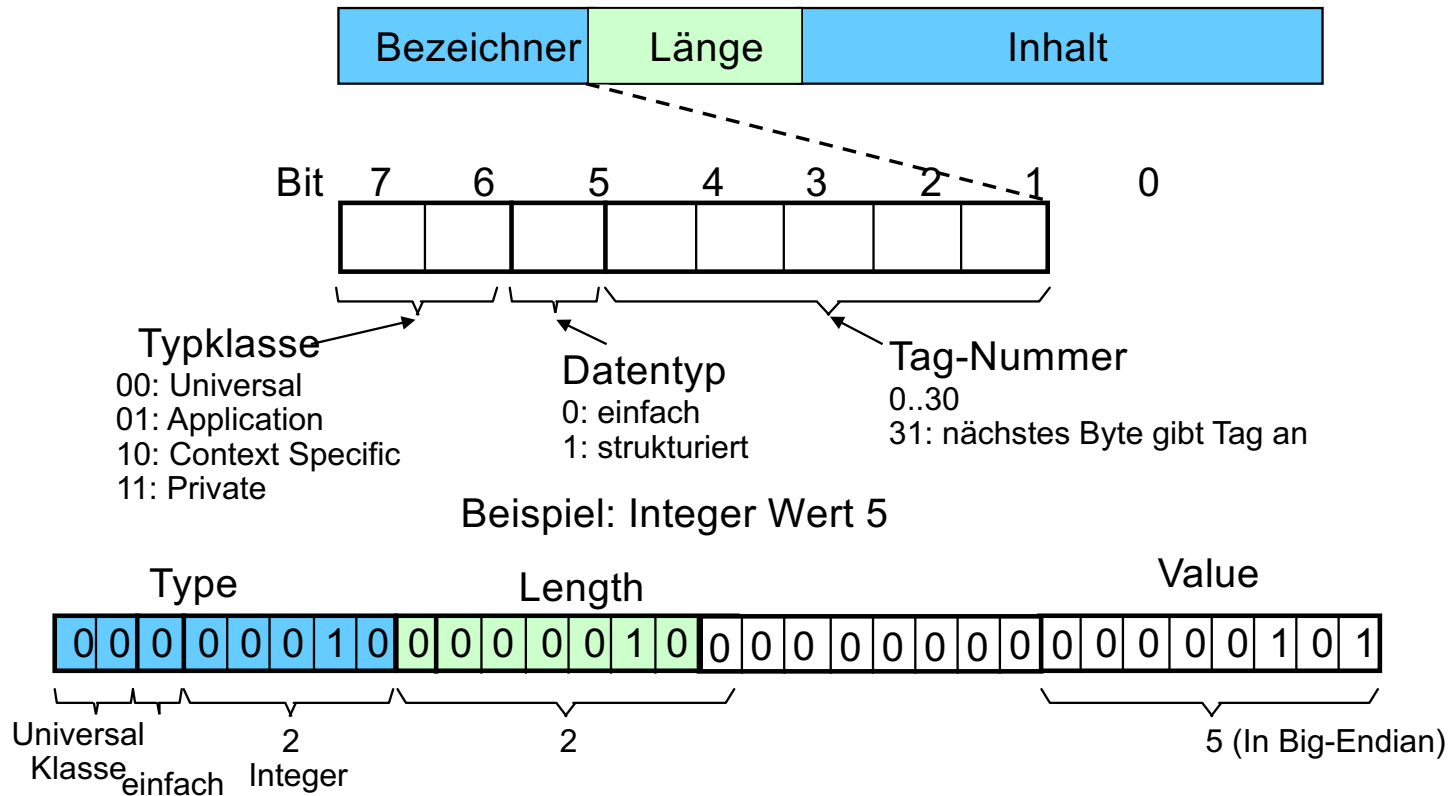
Beispiel:

```
Mitarbeiter ::= Set {  
    Name IA5String,  
    Alter Integer,  
    Personalnr Integer }
```

ASN.1: Übertragungssyntax

http://en.wikipedia.org/wiki/Distinguished_Encoding_Rules#BER_encoding

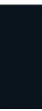
Zusätzlich zur Bereitstellung einer Datenbeschreibungssprache bietet ASN.1 auch sogenannte **Basic Encoding Rules**, also Regeln, die spezifizieren, **wie ASN.1-Objekte über das Netzwerk versendet werden** sollten.





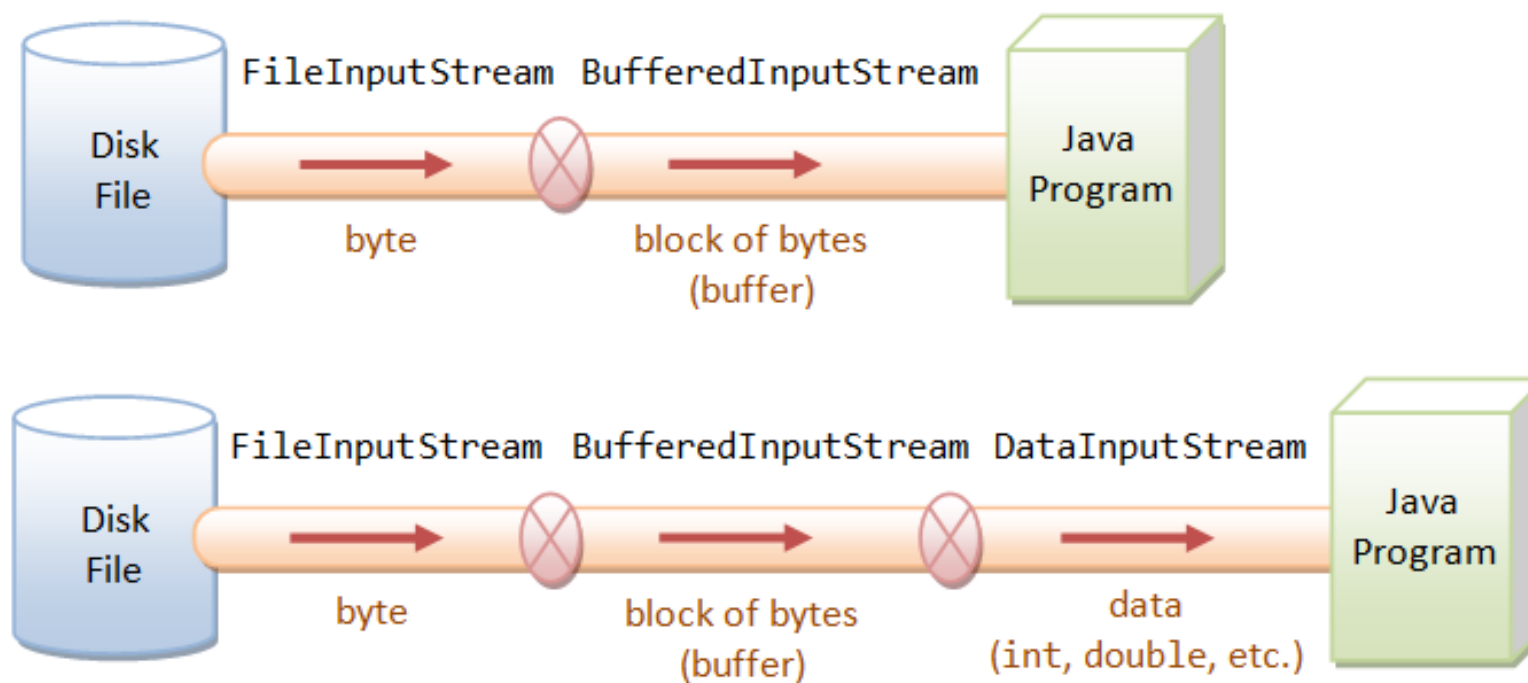
Objektserialisierung in Java

- Die Klasse **ObjectInputStream** bietet die Möglichkeit, mit der Methode `readObject()` serialisierte Objekte aus dem darunterliegenden Stream zu lesen
- Serialisierung über Objekt-Streams erlaubt das **Lesen und Schreiben von Objekten in einer Operation**. Serialisierung ist nicht auf singuläre Objekte beschränkt, da diese ja wieder andere Objekte referenzieren können, die somit auch verfügbar sein sollten. Faktisch kann also ein Objekt-Graph gelesen verarbeitet werden
 - `writeObject()` wandelt ein Objekt in einen Byte-Stream um
 - `readObject()` wandelt einen Byte-Stream in ein Objekt um
- Im Unterschied zum allgemeinen I/O- bzw. Streaming-Konzept beruhen diese Streams auf Interfaces. Somit ist auch eine komplett eigencodierte Serialisierung möglich (was einige Rahmenwerke auch so machen)





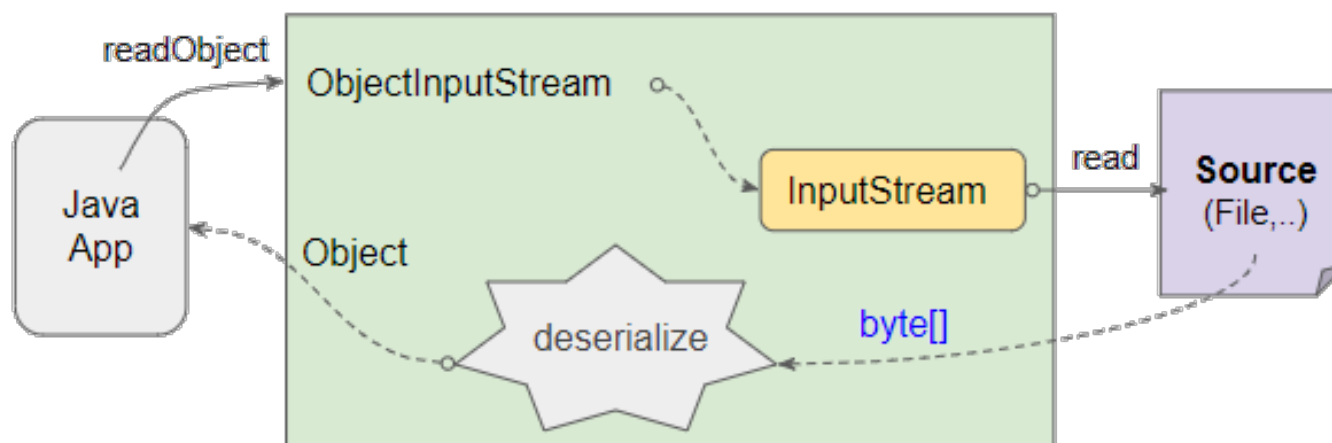
Objektserialisierung in Java



https://www3.ntu.edu.sg/home/ehchua/programming/java/J5b_IO.html



Objektserialisierung in Java



<https://o7planning.org/13397/java-objectinputstream>



Objektserialisierung mittels eXtensible Markup Language

- XML ist eine Metasprache zur Strukturierung von Dokumenten und Daten
- Basis von XML sind **beliebige Auszeichnungselemente**, die geringen syntaktischen Regeln zu folgen haben
 - Wohlgeformte XML-Dokumente (**DOM-Baum**)
- XML macht erst wirklich Sinn, wenn es in einer Anwendung konkretisiert wird. XML-Anwendungen ergeben sich aus
 - **Namensräumen**
Festlegen der semantisch logischen Bedeutung bei Homonymen in den Auszeichnungselementen (analog zu Packages in Java)
 - **Stylesheets**
Definition des Erscheinungsbildes von Auszeichnungselementen (hier unwichtig)
 - **Schemata**
Beschreiben die Dokumentenstruktur eines bestimmten Typs (wichtig)
- Beispiel: XSD





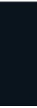
XML-Dokument, Beispiel

```
<?xml version="1.0" encoding="UTF-8"?>
<rechnung kundennummer="k4711">
  <monatspreis>0,00</monatspreis>
  <einzerverbindungsnachweis>
    <verbindung>
      <datum>26.2.</datum>
      <zeit>19:47</zeit>
      <nummer>200xxxx</nummer>
      <einzelpreis waehrung="Euro">0,66</einzelpreis>
    </verbindung>
    <verbindung>
      <datum>27.2.</datum>
      <zeit>19:06</zeit>
      <nummer>200xxxx</nummer>
      <einzelpreis waehrung="Euro">0.46</einzelpreis>
    </verbindung>
    <verbindungskosten_gesamt
      waehrung="Euro">2.19</verbindungskosten_gesamt>
    </einzerverbindungsnachweis>
  </rechnung>
```



XML-Dokument, Eigenschaften

- XML-Dokumente enthalten Daten und über die Auszeichnungselemente Strukturinformation über die Daten in einem Dokument
- Die Semantik ist aber nicht festgelegt
- XML-Dokumente haben (geringe) Strukturvorgaben (Wohlgeformtheit)
- Informationen (Inhalte) im XML-Dokument haben einen Datentyp (typisiert)
- XML ist textbasiert





Anatomie von XML-Dokumenten

- XML-Dokumente: Zeichendaten und Auszeichnungen

- XML- und Dokumenttyp-Deklarationen

- ```
<?xml version="1.0" encoding="UTF-8"?>
```

- Elemente mit möglichem Inhalt

- ```
<name>Volker Sander</name>
```

- Attribute in Elementen

- ```
<name attribute="Wert">
```

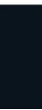
- Entity-Referenzen (&lt; statt <)

- Kommentare

- ```
<!-- Das ist ein Kommentar-->
```

- Processing Instructions – werden an die aufrufende Instanz weitergeleitet

- ```
<?name pidata?>
```

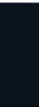




## Anatomie von XML-Dokumenten

---

- Jedes Dokument entspricht einer Baumstruktur (DOM)
- Die Baumknoten werden Elemente genannt
- Es kann nur ein Element an der **Wurzel** geben
- Syntaxregeln müssen strikt eingehalten werden
  - Jede Auszeichnung muss ein Endtag haben  
`<BOOK>...</BOOK>`  
`<BOOK />`
  - Verschachtelung nur mit vollständigen Tags möglich:  
`<BOOK> <LINE> This is OK </LINE> </BOOK>`  
`<LINE> <BOOK> This is </LINE> definitely NOT </BOOK> OK`
  - Attributwerte müssen mit ‘ oder “ abgegrenzt werden





---

# Wann ist ein XML-Dokument „korrekt“?



# Korrektheitsbegriffe

---

## Wohlgeformtheit

Dokument entspricht den syntaktischen Regeln

- Genau ein Dokument-Element
- Jedes öffnende Tag hat ein schließendes Tag
- Die Verschachtelung ist balanciert

Gegenbeispiel:

`<A><B><C> ... </A> ... </C>`

## Validität

Dokument ist

- wohlgeformt
- konform zu einer fest **vorgegebenen Dokumentenstruktur** ist

Document Type Description (DTD)

XML Schema Definition (XSD)





## Namensräume in XML: Namespaces

---

- Zur Erklärung eines XML-Schemas müssen wir erst ein anderes Problem diskutieren:

Zum Test eines Dokumentes gegen eine Strukturvorgabe müssen Elementbezeichnungen verarbeitet werden. Gleiche Elementbezeichnungen können jedoch unterschiedliche Bedeutung haben – **die Semantik ergibt sich also aus der Anwendung oder dem Kontext!**

```
<Schloss>Neuschwanstein</Schloss>
```

```
<Schloss>Sicherheitsschloss Typ X</Schloss>
```

- Ähnlich wie Variablen in Programmiersprachen haben Elementbezeichnungen einen Kontext/Scope
- Namespaces dienen der Trennung der Bedeutung der Bezeichner und ermöglichen so, Elementbezeichnungen und Inhalte eindeutig zu referenzieren



# Namensraumdeklaration

- Namensräume werden weltweit eindeutig durch eine **URI** (Uniform Resource Identifier) identifiziert
- Ein Präfix stellt eine verkürzte Notation bereit
- Namensraum-Präfix muss ein XML-Bezeichner sein
  - darf **keine Sonderzeichen** usw. enthalten
  - URIs beinhalten fast immer Sonderzeichen (<http://test.com/namespace>)
- Namensraumdeklaration ordnet Präfix und URI einander zu
- xmlns steht für XML Namespace und ist ein festgelegtes Attribut, das direkt von XML-Spezifikationen definiert wird und einen Mechanismus zur Bindung von Präfixen an URIs liefert

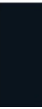
```
<person xmlns:job="http://www.berufe-online.de/berufe">
 <vorname>Carl Friedrich</vorname>
 <nachname>Gauß</nachname>
 <job:berufsbezeichnung>Mathematiker</job:berufsbezeichnung>
</person>
```



## XML Schema

---

- Ein XML-Schema ist eine XML-Anwendung, mit der die Struktur einer Klasse von XML-Dokumenten beschrieben werden kann
- Der Zweck eines XML-Schemas entspricht dem seines Vorgängers, dem DTD (welches nicht XML war)
- Ein XML-Schema enthält **in XML notierte Regeln, die erlaubte Elementbezeichner, Reihenfolgen, Inhalte und Attribute mit definierten Wertebereichen aufzählen**
- Es existieren vielfältige vordefinierte Datentypen, die Möglichkeit zur Definition eigener Datentypen und die eine Darstellung komplexer Integritätsbedingungen





## XML Schema

---

- Um ein Dokument validieren zu können muss es das seine Struktur beschreibende **Schema referenzieren** (hierzu ist es nicht verpflichtet). Dies kann für **einfache** Schemata mittels **noNamespaceSchemaLocation** geschehen, für komplexere mit mehreren Namensräumen verwendet man **SchemaLocation**

```
<?xml version="1.0" ?>
```

```
<artikel xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:noNamespaceSchemaLocation="artikel_simple.xsd">
```

- Das Schema-Dokument selber nutzt den **Schema-Namespace** und wird meistens durch die Präfixe **xs:** oder **xsd:** gekennzeichnet

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
```

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
```





# Ein XML-Schema

Beispiel: `<datum jahr="1979"/>`

Namensraumdeklaration für  
XML-Schema

```
<xsd:schema xmlns:xsd="http://www.w3c.org/2001/XMLSchema">
 <xsd:element name = "datum" type= "datumsTyp"/>
 <xsd:complexType name= "datumsTyp">
 <xsd:attribute name = "jahr">
 <xsd:simpleType>
 <xsd:restriction base = "xsd:integer">
 <xsd:maxInclusive value = "2500">
 <xsd:minInclusive value = "0">
 </xsd:restriction>
 </xsd:simpleType>
 </xsd:attribute>
 </xsd:complexType>
```

Element-  
Deklaration

Typdefinitionen

Attribut-  
Deklaration



## Elementdeklarationen

---

- Neben der Benennung des erwarteten Auszeichnungselements wird auch der Datentyp angegeben werden (per Referenz oder direkter Auflistung), der beim Inhalt des Elementes erlaubt ist

- **Primitive Datentypen**

**Schema:** `<element name="float-element" type="float" />`

**Dokument:** `<float-element>123.456 </float-element>`

- **Deklaration von Datentypen (Elementen)**

`<complexType name="studenttype"> ... </complexType>`

- **Ableitung existierender Datentypen mittels Angabe eines Musters (Mit Großbuchstaben beginnend, dann ein oder mehrere Zahlen)**

`<simpleType name="studentid" base="string">  
 <pattern value="[A-Z][0-9]+" />  
</simpleType>`





# Einfache Datentypen

---



Beispiele für Elementdeklarationen mit einfachen Datentypen:

```
<xsd:element name="name" type="xsd:string"/>
<xsd:element name="schuhgroesse"
 type="xsd:positiveInteger"/>
<xsd:element name="geburtsdatum" type="xsd:date"/>
```

Eine entsprechend valide XML-Datei:

```
<name>Carl Friedrich Gauß</name>
<schuhgroesse>42</schuhgroesse>
<geburtsdatum>1777-04-30</geburtsdatum>
```



## Komplexe Typen

---

- Neben diesen eingebauten Datentypen bietet ein XML-Schema die Möglichkeit, eigene Datentypen abzuleiten
- Komplexe Typen benötigt man immer dann, wenn man Elemente deklarieren möchte, die Attribute besitzen oder andere Elemente beinhalten können, denn folgendes ist so nicht möglich:

```
<xs:element name="Kunde">
 <xs:element name="Vorname" type="xs:string">
 <xs:element name="Zweitname" type="xs:string">
 <xs:element name="Nachname" type="xs:string">
</xs:element>
```





# Komplexe Typen

---

- Erfolgt die Definition eines komplexen Typs für ein Element innerhalb des `<xs:element>`-Elements, so spricht man von einem **anonymen Typen**

```
<xs:element name="Kunde">
 <xs:complexType>
 <xs:sequence>
 <xs:element name="Vorname" type="xs:string"/> <xs:element
name="Zweitname" type="xs:string"/> <xs:element name="Nachname"
type="xs:string"/>
 </xs:sequence>
 </xs:complexType>
</xs:element>
```





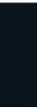
# Komplexe Typen

---

- Sollen mehrere Elemente die gleichen Kindelemente und Attribute tragen, so sollten **benannte komplexe Typen** benutzt werden
- Diese werden über das type-Attribut in der Element-Deklaration zugewiesen

```
<xs:complexType name="KundenTyp">
 <xs:sequence>
 <xs:element name="Vorname" type="xs:string"/>
 <xs:element name="Zweitname" type="xs:string"/> <xs:element
name="Nachname" type="xs:string"/>
 </xs:sequence>
</xs:complexType>

<xs:element name="Kunde" type="KundenTyp">
```





# Werkzeuge zur Bildung komplexer Typen

---

Kompositoren

Es gibt drei Kompositoren, die kombiniert werden können





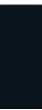
# XML – Schema (sequence)

---

Sequenzen:

Die Inhalte einer Sequenz müssen in der **Reihenfolge** erscheinen, in der sie angegeben werden:

```
<xsd:sequence>
 [Inhalt 1]
 ...
 [Inhalt n]
</xsd:sequence>
```





# XML – Schema (all)

---

## Die **all**-Gruppe:

Alle Elemente, die in der **all**-Gruppe deklariert sind, dürfen in **beliebiger Reihenfolge** instanziiert werden, aber höchstens einmal.

Außerhalb der **all**-Gruppe darf es keine weiteren Element-Deklarationen geben.

```
<xsd:all>
 [Elementdeklarationen]
</xsd:all>
```





## XML – Schema (choice)

---

Auswahllisten:

Von den aufgeführten Inhalten muss ein frei wählbarer instanziiert werden:

```
<xsd:choice>
 [Inhalt a]
 ...
 [Inhalt x]
</xsd:choice>
```





## Ergänzungen zu den Kompositoren

---

Auswahllisten und Sequenzen können beliebig geschachtelt werden.

Neben der Anordnung kann auch die Häufigkeit festgelegt werden, mit der Elemente erscheinen dürfen.

**minOccurs** gibt an, wie oft ein Element mindestens erscheinen muss (default 1; Angabe optional).

**maxOccurs** gibt an, wie oft ein Element höchstens auftreten darf (default 1;  $\infty$  = unbounded; Angabe optional).

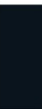
**default** gibt den Wert an, der dem Element zugeordnet wird, wenn es vorhanden, aber leer ist.

```
<element name="vorstandsmitglied" minOccurs=4 maxOccurs=6>
```



# Abgeleitete Typen

---



Ein abgeleiteter Datentyp wird auf der Basis eines anderen Datentyps durch Einschränkung oder Erweiterung spezifiziert. Auch von abgeleiteten Datentypen können weitere Datentypen abgeleitet werden.

Datentypen können in verschiedener Weise abgeleitet werden:

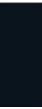
- durch Einschränkung des Wertebereichs (restriction)
- über eine durch Whitespaces notierte Liste verschiedener Werte (list)
- über eine Vereinigung mehrerer Typen (union)
- durch Erweiterung (nur komplexe Typen) (extension)



# Objekt Mappings

---

- XML-Schema beschreiben eine Klasse von Dokumenten
- XML-Dokumente sind somit eine Instanz einer Klasse von Dokumenten
- Dies ermöglicht parallelen zur objektorientierten Welt:
  - Der Zustand eines Objektes kann in einem XML-Dokument festgehalten werden
  - Die Struktur der Klasse liefert das Schema
- Die Grenzen zwischen XML, Strukturen in Programmiersprachen und Datenbanken verschwimmen
- Viele Tools sind bereits verfügbar
  - LINQ2XSD .NET
  - Hibernate / JAXB / Java Persistence API



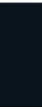


# Objekt Mappings

---

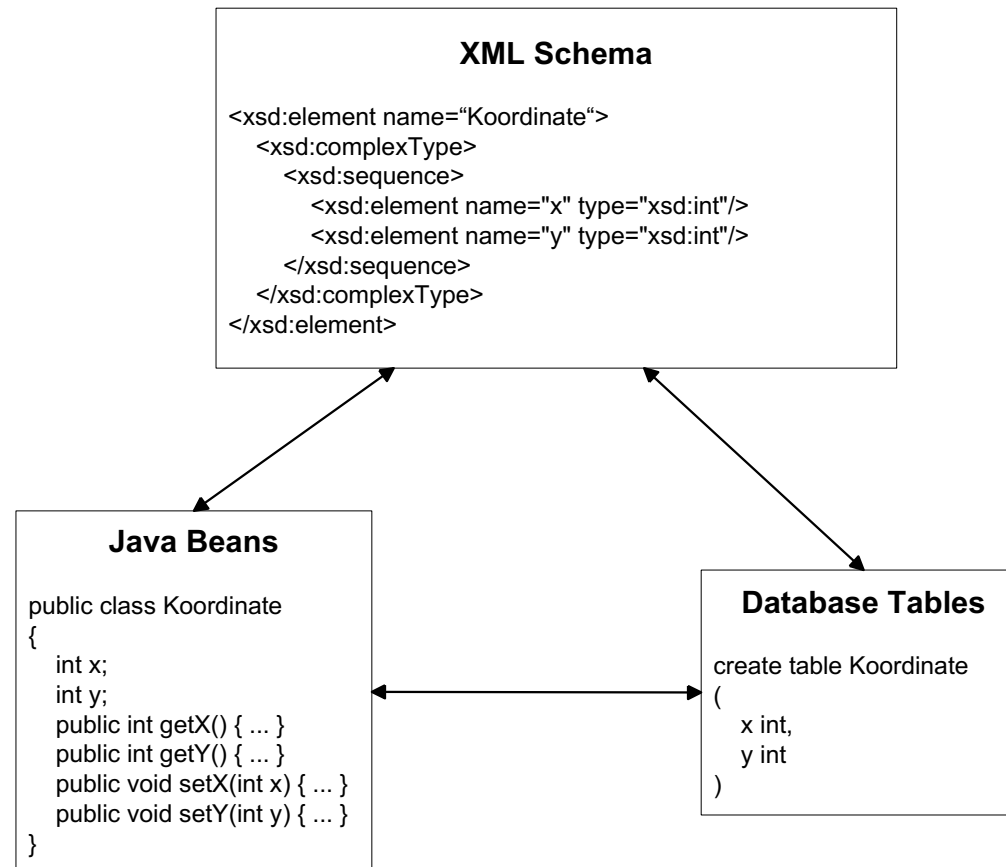
Mittels einer xsd-Datei können Schnittstellen sehr gut beschrieben werden. Über JAXB können beispielsweise Java-Klasse generiert werden, die beim Marshalling und Unmarshalling der validen XML-Dateien genutzt werden können. Hierfür gibt es dann auch eine Marshaller und Unmarshaller Klasse

Umgekehrt kann man über die `@XmlRootElement`-Annotation aus Java-Klassen ein XML-Schema erzeugen



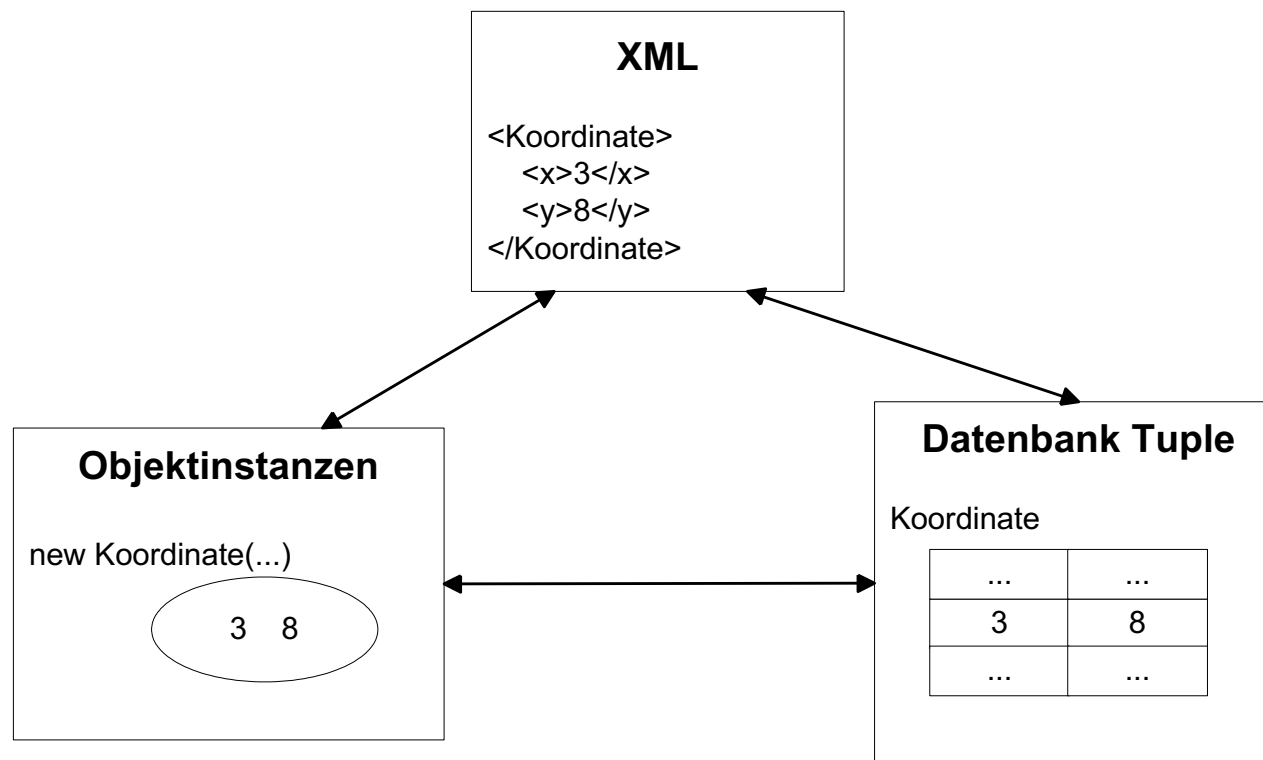


# Mapping auf der Schema-Ebene





# Mapping auf der Instanz-Ebene





# JSON

## JSON: JavaScript Object Notation

- JSON-Dokumente sind gültiges JavaScript
- Einfache Struktur
- Typisierung eingebaut!
- Beispiel:

```
{
 "id" : 2648,
 "Name" : "Mustermann",
 "Vorname" : "Max",
 "adr" : {
 "Stadt" : "Aachen",
 "plz" : 52064
 },
 "tel" : ["0241 1234", "0160123456"],
 "partner" : null,
 "maennlich" : true
}
```

Weitere Informationen: <http://www.json.org/>





## JSON in Java mittels *org.json*

### Objektserialisierung:

```
DemoBean demo = new DemoBean();
```

```
demo.setId(1);
```

```
demo.setName("lorem ipsum");
```

```
demo.setActive(true);
```

```
JSONObject jo = new JSONObject(demo);
```

```
{"name":"lorem ipsum","active":true,"id":1}
```

**Meistens nimmt man aber automatische Marshaller/Unmarshaller wie Jackson**





## Hinweis

---

XML und JSON sind textbasierte Objektserialisierungsformate für die Darstellungsschicht

Beide Formate lassen sich faktisch in allen Programmiersprachen einfach integrieren

Beide sind äußerst populär

XML hat aufgrund des Schemas einen Vorteil für typisierte Programmiersprachen

JSON-Schema ist ein de-facto Standard, mit dem dies ein wenig gemildert wird

